

Playing Atari with Deep Reinforcement Learning 2013

梗概

本文章是深度强化学习史上最重要的论文之一。第一次实现了从**原始像素（图像）到动作**（pixels -> actions）。第一次让 deep learning 与 Q-Learning 真正结合。为后面的 AlphaGo, AlphaZero, Rainbow等工作奠定基础。是现代 Deep RL 的起点之一。

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning.

该文章提出了第一个能够

- 使用强化学习
- 从高维感知输入
- 直接学习控制策略

的深度学习模型 **DQN** Deep Q-Networks 并应用了 **experience replay mechanism** 规避 RL + Deep Learning 的常见困难

以前的强化学习需要程序员手工提取特征。而作者想证明，对于视觉任务，可以直接使用 CNN 提取特征，经过 Q-Learning 学习然后做出动作

动机

作者认为强化学习的长期目标是通过 **视觉完成决策**，如同人类看到，思考，行动的反应链条。

而强化学习缺少泛化能力，即微小的状态差异对于 Tabular RL 就变成了完全不同的状态，需要重新学习。而 CNN 由于自动的特征提取的特性，具有天然的泛化性。于是作者想到将两者进行超级拼装！

而作者提出想要将 RL 与 Deep Learning 结合有三大困难

- **奖励稀疏**

RL algorithms ... must be able to learn from a scalar reward signal that is frequently sparse, noisy and delayed.

这就是我们在上一篇文章 **Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning** 中提到的 **奖励归因问题 Credit Assignment Problem**，即监督学习我们有样本和标签，但是在 RL 中有时奖励并非立即出现而是在多步之后才有（依然围棋举例，只有在下很多手到终局才知道谁获胜），因此其中的每一步对奖励有多大贡献就存在问题

- **样本相关**

one typically encounters sequences of highly correlated states.

监督学习假设样本 **独立同分布 (i.i.d.)**

 **Note**

独立同分布 (Independent and Identically Distributed, 简称 **I.I.D.**)

一组随机变量之间**互不影响 (独立)**，并且它们都**服从同一个规律 (同分布)**

但是在 RL 中，连续的状态高度相似，这样对于 SGD 来说

- 方差数变大，有效样本数量下降
- 容易震荡
- **数据分布不断变化**

监督学习训练集是固定的，而强化学习变化策略后数据也会变化

To alleviate the problems of correlated data and non-stationary distributions, we use an experience replay mechanism.

为了避免这些困难，作者提出了 **experience replay mechanism**

作者认为深度网络训练失败不是因为 CNN 不够强，而是训练数据有问题。需要打乱经验，变成近似的 i.i.d. 分布

为什么能用 RL 玩 Atari

优化对象

标准 RL 理论建立在 s_t 是真实状态的前提下，但是对于 Atari 的游戏来说有一个问题，agent 能看到的只有屏幕截图，只能获取截图中已有的状态信息，而无法读取玩家游戏内部变量。

因此，作者特别的使用了一个术语 “perceptually aliased”：允许不同真实状态产生相同的画面

为了适配这个特性，作者对状态进行了重定义

$$s_t = x_1, a_1, x_2, \dots, a_{t-1}, x_t$$

即 agent 的完整历史，这样原本是 POMDP (Partially Observable MDP) 的 Atari 就获得了马尔可夫性 (当前状态浓缩了预测未来所需的全部信息，数学上 $P(S_{t+1}|S_t) = P(S_{t+1}|S_t, S_{t-1}, \dots, S_0)$)，可以继续使用 Q-Learning

优化目标

接下来还需要定义 “什么是好策略”。

作者定义了 **折扣回报 Discounted Return**，在 t 时的折扣回报

$$R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$$

它表达的是，从当前开始未来所有奖励总和， γ 控制未来奖励的重要程度如果为 0 则只关心眼前，如果为 1 则只关心最终结果

未来收益属于整条轨迹，而agent决策的时候面对的是当前动作，所以作者需要在动作和未来收益之间建立一个桥梁，借鉴了 Q-Learning 的思想，作者定义

$$Q^*(s, a) = \max_{\pi} E[R_t | s_t = s, a_t = a, \pi]$$

- s 是一系列状态
- a 是一系列动作
- π 是策略，即 policy mapping sequences (or distributions) to (over) actions

于是最大化未来收益目标就转化为了估计动作价值

这个形式仍然不便于计算，我们期望将其转化成一种迭代的形式这样我们就可以用解决 DP 问题一样求解

Bellman 给出的方案是：只看下一步

if the optimal value at the next time-step was known ...

当前动作的价值 = 当前奖励 + 未来最好价值：

$$r + \gamma \max_{a'} Q^*(s', a')$$

这样未来收益就通过递归的方式转变成一个动态规划问题（如果有些疑问可以参考 Q-Learning notes中的讲解）

接下来我们就可以获得一个递归形式的通式

许多强化学习算法的基础思路就是将 Bellman equation 作为递归迭代去估计 action-value function

$$Q_{i+1}(s, a) = E[r + \gamma \max_{a'} Q_i(s', a') | s, a] \quad (1)$$

当 $i \rightarrow \infty$ 时 $Q_i \rightarrow Q^*$

Bellman 将 **公式 (1)** 的右边这一串定义成了算子 (**Bellman Operator**) T ，那么迭代公式就能改写为

$$Q_n = T^{n-1} Q_0$$

显然最优的 Q^* 是一个 **不动点 fixed point** 有性质

$$Q^* = TQ^*$$

作者对这个 **等式 (1)** 提出了简单一个求解思路

1. 随机初始化一个 Q_0
2. Bellman 更新
3. 得到更好的 Q
4. repeat 2, 3

例如

$$\begin{aligned} & \text{Random } Q_0 \\ Q_1 &= E[r + \gamma \max Q_0] \\ Q_2 &= E[r + \gamma \max Q_1] \\ & \dots \end{aligned}$$

有朋友要问了，这样随机初始化也能收敛吗？是的，**Bellman Operator 是压缩映射**

$$\|TQ_1 - TQ_2\| \leq \gamma \|Q_1 - Q_2\|$$

因为 $\gamma < 1$ 每更新一次误差就缩小 γ 倍，在迭代次数足够多时 Q 就能无限趋近于 Q^* ，

数学你就学吧，学无止境

作者说 Bellman 说的对啊，但是！这个方法在 Atari 的游戏上根本没法用。

Bellman 提出的方法实际上就是 Q Table，正如我们在前文提到的，在人类看来相似的状态对于 Q Table 来说却是完全不同的状态，缺少泛化，而且状态空间在无限大的情况下，Q Table 完全无法储存。于是作者在 Q 值的定义上动了一点小巧思

$$Q^*(s, a) \text{ replaced by } Q(s, a; \theta)$$

我们来到了二战转折点，作者使用了 θ 代表神经网络的所有参数，原本 Q 值依赖查表获得，而现在直接由神经网络计算，之后更新也不是对 Q 值，而是更新参数 θ

构造 Loss

神经网络究竟怎么训练？

对于监督学习来说标签已知，但是对于 Q 值呢？最优 Q 值本来就是未知的。作者找回了 Bellman Equation

$$Q^*(s, a) = E[r + \gamma \max_{a'} Q^*(s', a')] \quad (2)$$

利用其自己制造标签

作者发现 Q^* 满足 Bellman 方程，那么如果当前网络已有了一个 Q 的估计 $Q(s, a; \theta)$

就可以构造

$$y_i = r + \gamma \max_{a'} Q(s', a'; \theta_{i-1})$$

作为训练目标，那么 Loss function 就可以使用 MSE 的形式被表示为

$$L_i(\theta_i) = E[(y_i - Q(s, a; \theta_i))^2] \quad (3)$$

细心的小伙伴会发现 y_i 跟 θ_{i-1} 有关但是 **公式 (3)** y_i 又是和 θ_i 的 Q 做差，作者是怎么解释的

The parameters from the previous iteration θ_{i-1} are held fixed when optimising the loss function.

为了稳定，我们需要让前一轮的参数固定

Loss Function 的梯度

接下来我们就可以到达轻松愉快的求梯度时间了

$$\nabla_{\theta_i} L_i(\theta_i) = E_{s,a \sim \rho(\cdot), s' \sim \mathcal{E}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right] \quad (4)$$

依旧是经典的 误差 $\times$ 导数 (作者偷懒了省略了前面的常数系数 -2)

对于公式中的这一坨

$$r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)$$

这篇文章的作者没有起名，但是后来的强化学习领域给了一个名字 **TD ERROR (temporal difference error)** 记作 δ

相信读者已经发现了，**公式 (4)** θ_i 进行了求导，而没有对 θ_{i-1} 进行求导。回顾上文，构造 loss 的时候我们将 θ_{i-1} 固定成常数了，所以本来应该会出现 $\nabla_{\theta} y$

$$\nabla_{\theta} L(\theta) = E [2\delta(\theta) (\nabla_{\theta} y(\theta) - \nabla_{\theta} Q(s, a; \theta))]$$

由于为 0，作者故意忽略，只保留了 $\nabla_{\theta} Q$ 只求了一半梯度因此叫 **semi-gradient**

作者依然不满意，因为这个梯度里面还有期望 E ，而期望意味着需要遍历所有状态，显然这违背了作者希望在无限状态环境中使用 RL 的主旨，因此作者讲到：

Rather than computing the full expectations... it is often computationally expedient to optimise the loss function by stochastic gradient descent.

不计算期望了，直接采一个样本就更新一次，用单个样本近似

这样我们就获得了 SGD 版本的 Q-Learning

$$Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \rightarrow Q(s, a)$$

SGD 版 Q-Learning 的特征

- **model-free**

不用学习环境模型，不显式建模 $P(s'|s, a)$ 或 $R(s, a)$

- **off-policy**

模型学习的是 greedy strategy，但采样时遵循的是 exploration behavior distribution

也就是说，训练目标想学的是

$$\pi_{target}(s) = \arg \max_a Q(s, a)$$

但是实际收集数据时用的是 $\pi_{behavior}$ 比如 ϵ -greedy 在上一篇讲解 Simple Statistical Gradient-

Following Algorithms for Connectionist Reinforcement Learning 的 note 中有更加详细一些的解释

相关工作

TD-Gammon 的成功与失败

TD-Gammon (1995) 也实现了神经网络 + 强化学习 + 自我博弈，在 TD-Gammon 成功后大家立刻尝试 国际象棋，围棋，跳棋，结果遭遇失败，学界形成了一种观点认为 TD-Gammon 只是运气好或者 Backgammon 比较特殊，但是作者指出：

perhaps because the stochasticity in the dice rolls helps explore the state space and also makes the value function particularly smooth

backgammon 使用了骰子，随机性带来了天然的 **探索 exploration** 性，同时让价值函数更加平滑因此神经网络更容易拟合

发散问题

当时理论界认为如果同时满足：

- Q-Learning
- Nonlinear Function Approximation
- Off-policy

那么模型的训练无法收敛，直接崩溃。后世称这三元素为 **Deadly Triad**

改用线性模型

因为神经网络总是发散，大家退而求其次继续沿用 人工特征 + 线性模型的组合

NFQ: DQN 之祖

Perhaps the most similar prior work to our own approach is neural fitted Q-learning (NFQ). NFQ optimises the sequence of loss functions in Equation 2, using the RPROP algorithm to update the parameters of the Q-network.

NFQ Neural Fitted Q-learning 2005，作者承认 NFQ 的工作和本文有一些相似之处，但是作者同样指出：

However, it uses a batch update that has a computational cost per iteration that is proportional to the size of the data set, whereas we consider stochastic gradient updates that have a low constant cost per iteration and scale to large data-sets.

- **NFQ 需要 full Batch training 而 DQN 可以抽样训练和更新**
- 以前的做法是将图像给 AutoEncoder 处理成低维特征，再交给 RL 这样分成两阶段，先学表示，再学策略。**DQN 则是将图像直接丢给 Q network 然后得到 Q 值，一步完成，无需像 AutoEncoder 一样重建背景。**

Experience Replay

经验回放不是 DQN 发明的，而是引用了 Long-Ji Lin 1993 的工作，DQN 的创新是将这些工作成功整合在了一起，这在当年是极其震撼的

Deep Reinforcement Learning

这一部分作者开始更多的从工程上是怎么将 related work 中的技术有效的整合在一起的

作者观察到在 2012 年前后

- AlexNet 成功
- ImageNet 成功
- 语音识别成功

他们都使用了海量数据 + SGD

同时回顾 TD-Gammon，其训练时，每一步得到样本就立刻更新网络，作者认为这个方案无法实现，因为 Atari 维度过大（84*84*4）连续样本高度相关，于是 online SGD 很容易崩

Experience Replay

在这里作者正式定义，作者将每一步经历以：

$$(s_t, a_t, r_t, s_{t+1})$$

的形式储存到经验池，后来这个四元组被称之为 **Transition** 转移样本

在训练的时候从经验池中随机抽取，组成 mini-batch 然后更新一次网络

作者给出三个理由，这些理由也成为后来 Experience Replay 被广泛使用的原因

- **提高样本利用率**

First, each step of experience is potentially used in many weight updates, which allows for greater data efficiency.

通俗的说就是经验可以被反复抽取，反复训练

- **打破相关性**

randomizing the samples breaks these correlations and therefore reduces the variance of the updates.

连续样本相关性强，SGD 效果变差

- 平滑数据分布

By using experience replay the behavior distribution is averaged over many of its previous states, smoothing out learning and avoiding oscillations or divergence in the parameters.

如果不使用 Replay 策略变化会导致数据分布变化，从而更新得到的策略变化会不稳定

Experience Replay 也存在局限性。作者采用了 FIFO Buffer 作为经验池，同时均匀随机采样

作者指出

the memory buffer does not differentiate important transitions... Similarly, the uniform sampling gives equal importance to all transitions.

很显然，Buffer 并没有区分哪些经验重要。因此作者预言

A more sophisticated sampling strategy might emphasize transitions from which we can learn the most, similar to prioritized sweeping.

优先学习重要经验（例如 TD Error 更大的经验），在2015年有人实现了作者的这一预言 Prioritized Experience Replay (PER) Tom Schaul

为什么不使用 SARSA

看到 SARSA Target

$$y_{SARSA} = R + \gamma Q(s', a')$$

这里的 a' 是实际执行的动作，因此 SARSA 学习的是 **当前行为策略** 即是 On-Policy，这会导致经验池中抽出的经验与当前策略会执行的动作不一致，从而导致 SARSA Target失真

预处理与模型结构

模型的输入是 210×160 并且有 128 种颜色

预处理

作者进行了两步操作

1. 灰度化

2. 缩放

原始 210×160 变成 84×84 ,

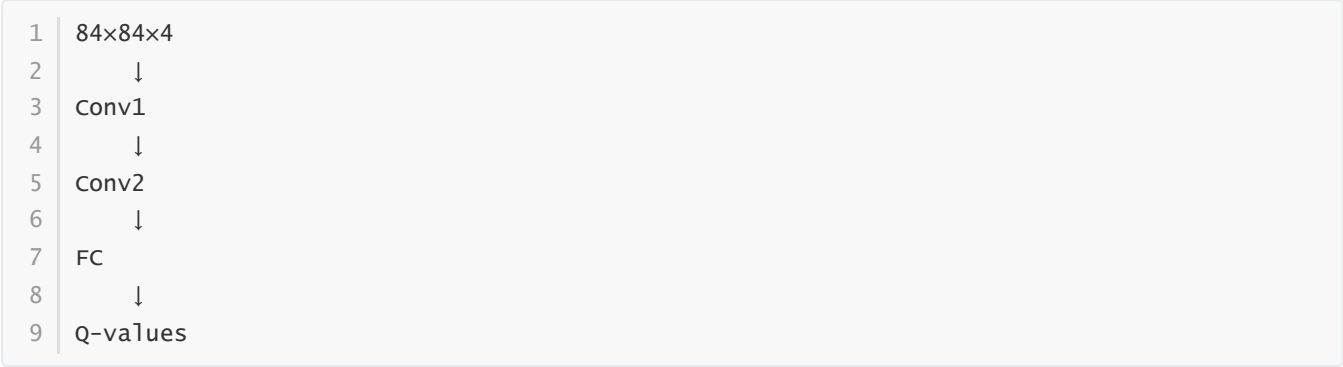
至于为什么要选 84，作者随便选的，只是因为足够小而且能够看清目标

为了使某一画面有上下文，作者采用最近四帧堆叠的方式，即 $s_t \approx (x_{t-3}, x_{t-2}, x_{t-1}, x_t)$

这样让 CNN 学习到速度和方向等信息，也是经验之选

于是经过预处理，这种 $84 \times 84 \times 4$ 的张量就是 CNN 的输入了

模型结构



模型结构还是相当的简单

- Conv 1 采用了 16 个大小为 8×8 步长为 4 的卷积核
采用这么大的核是因为 Atari 图像非常简单，增大核大小能一次看到更大区域提升处理速度
- Conv 2 采用了 32 个大小为 4×4 步长为 2 的卷积核
随着空间尺寸缩小，增加通道数学习更复杂的模式
- FC 采用了 256 个神经元，原文说
fully-connected and consists of 256 rectifier units
意思应该是 flatten 之后全连接到 256 维再经过了 ReLU （如有错误请指正）
- 输出层对于不同游戏维度不同，都表示动作得分

游戏	动作数
Pong	6
Breakout	4
Space Invaders	6
Seaquest	18

训练细节

数据

The behavior policy during training was ϵ -greedy with ϵ annealed linearly from 1 to 0.1 over the first million frames and fixed at 0.1 thereafter.

作者采用了 ϵ -greedy 策略

$$a_t = \begin{cases} \arg \max_a Q(s_t, a), & \text{with probability } 1 - \epsilon, \\ \text{random action}, & \text{with probability } \epsilon. \end{cases}$$

开始时 $\epsilon = 1$ 慢慢逐渐下降到 0.1，这是因为网络刚初始化 Q 值全是垃圾，如果直接贪心可能永远卡在局部区域，所以开始倾向于让模型随机探索

Replay Memory 大小

Buffer 储存了 100 万帧

Reward Clipping

不同 Atari 游戏奖励差异巨大，因此作者使用了一个简易的 clipping

$$r_{\text{clip}} = \begin{cases} +1, & r > 0, \\ 0, & r = 0, \\ -1, & r < 0. \end{cases}$$

作者在精确和稳定的 trade-off 中选择了稳定性

Eval 设计

- Baseline 1
Random Agent
完全随机，乱按一通得到 $Score_{\text{random}}$
- Baseline 2
Human Tester
得到 $Score_{\text{human}}$
- Baseline 3
诸如 SARSA, Linear RL, Neuroevolution

测试时将 ϵ 设为 0.05 即 95% 为贪心动作，5% 随机探索

结果分析

	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	−20.4	157	110	179
Sarsa	996	5.2	129	−19	614	665	271
Contingency	1743	6	159	−17	960	723	268
DQN	4092	168	470	20	1952	1705	581
Human	7456	31	368	−3	18900	28010	3690
HNeat Best	3616	52	106	19	1800	920	1720
HNeat Pixel	1332	4	91	−16	1325	800	1145
DQN Best	5184	225	661	21	4500	1740	1075

很多游戏上 DQN 的表现都超过人类，在 Breakout 上远超人类，一是因为打砖块游戏立刻获得奖励更有利于 Bellman 传播，Replay Buffer 也容易累计有效经验，二是 DQN 发现了极优的策略而一般玩家不易发现

相反的在 Montezuma's Revenge 上接近平均水平，原因是奖励极其稀疏。

这正暴露了 DQN 的最大弱点，即适合 dense reward 而不适合 sparse reward

最后附上 DQN 算法的伪代码流程

Algorithm 1 Deep Q-learning with Experience Replay

```
1: Initialize replay memory  $\mathcal{D}$  to capacity  $N$ 
2: Initialize action-value function  $Q$  with random weights
3: for episode = 1, ...,  $M$  do
4:   Initialize sequence  $s_1 = \{x_1\}$ 
5:   Preprocess state  $\phi_1 = \phi(s_1)$ 
6:   for  $t = 1, \dots, T$  do
7:     With probability  $\epsilon$ , select a random action  $a_t$ 
8:     Otherwise select

$$a_t = \arg \max_a Q(\phi(s_t), a; \theta)$$

9:     Execute action  $a_t$ 
10:    Observe reward  $r_t$  and image  $x_{t+1}$ 
11:    Set

$$s_{t+1} = (s_t, a_t, x_{t+1})$$

12:    Preprocess

$$\phi_{t+1} = \phi(s_{t+1})$$

13:    Store transition

$$(\phi_t, a_t, r_t, \phi_{t+1})$$

    in  $\mathcal{D}$ 
14:    Sample random minibatch of transitions

$$(\phi_j, a_j, r_j, \phi_{j+1})$$

    from  $\mathcal{D}$ 
15:    Compute target

$$y_j = \begin{cases} r_j, & \phi_{j+1} \text{ terminal} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta), & \text{otherwise} \end{cases}$$

16:    Perform a gradient descent step on

$$(y_j - Q(\phi_j, a_j; \theta))^2$$

17:  end for
18: end for
```
